PROGRAMAÇÃO EM C PARA UNIX

DEIS – Sistemas Operativos – Trabalho Prático – 2025/2026

Regente da cadeira:

João António Pereira Almeida Durães – jduraes@isec.pt

Realizado por:

Pedro Saraiva – a2023146226@isec.pt
Rúben Costa – a2024132692@isec.pt

Índice

1. Desenhar a arquitetura da solução. 2
1.1. Contexto Geral da Disciplina 2
2. Arquitetura do sistema 3
2.1. Processos e responsabilidades 3
2.2. Canais de comunicação 4
3. Estruturas internas e modelo de estado 5
4. Implementação 6
4.1. Organização do projeto 6
4.2. Concorrência no controlador: threads reais 6
4.3. Mutexes: onde protegem e o que foi evitado 7
4.4. Variável de ambiente NVEICULOS 7
5. Tabela de funcionalidades requeridas 8
6. Conclusões 9

1. Desenhar a arquitetura da solução.

1.1. Contexto Geral da Disciplina

Este trabalho implementa uma plataforma de transporte com três executáveis: controlador, cliente e veículo. O objetivo é permitir que vários utilizadores (vários processos cliente) façam login, agendem, consultem e cancelem serviços; que o controlador gere a frota e lance veículos à hora agendada; e que cada veículo simule uma viagem com progresso em percentagem, permitindo ao utilizador entrar, sair a meio, e lidar com cancelamentos por SIGUSR1.

O enunciado exige ainda restrições importantes:

- comunicação com cliente via named pipes;
- comunicação controlador→veiculo via argumentos e veiculo→controlador via stdout redirecionado / pipes anónimos;
- sem select() no controlador;
- no máximo um processo controlador em execução;
- dimensão da frota definida pela variável de ambiente NVEICULOS.

2. Arquitetura do sistema

2.1. Processos e responsabilidades

Controlador (servidor):

- recebe pedidos dos clientes por FIFO público (named pipe);
- mantém a lista de utilizadores ativos e a lista de serviços (estado, hora, distância, etc.);
- simula "tempo" com um contador (hora simulada);
- à hora certa, lança processos veículo via fork() + exec();
- recebe telemetria dos veículos pela saída standard redirecionada para um pipe anónimo;
- mantém um contador global de km totais do sistema.

Cliente:

- é a interface do utilizador (linha de comandos);
- comunica com o controlador por named pipes;
- quando recebe do veículo a mensagem "cheguei ao local", passa a comunicar diretamente com o veículo (também por named pipe) para entrar <destino> e sair.

Veículo:

- é lançado pelo controlador na hora do serviço;
- recebe info do serviço via argumentos;
- avisa o cliente que "chegou ao local" e fornece o FIFO do próprio veículo para o cliente enviar entrar;
- simula deslocação a 1 km por unidade de tempo e reporta PROGRESSO de 10% em 10% no stdout;
- aceita comando sair durante a viagem e termina o serviço;
- ao receber SIGUSR1 cancela o serviço, informa o cliente e termina.

2.2. Canais de comunicação

(A) Cliente ↔ Controlador

- FIFO público (do controlador): usado pelos clientes para enviar pedidos.
- FIFO privado por cliente: usado para o controlador responder a cada cliente. Este modelo segue o padrão dado nas aulas: um canal comum de pedidos e um canal privado de resposta por cliente, evitando colisões de respostas.

(B) Controlador ↔ Veículo

- Controlador → Veículo: **argumentos de linha de comandos** (id, distância, local, username, PID cliente e FIFO cliente).
- Veículo → Controlador: mensagens de estado pelo **stdout**, capturado por um **pipe anónimo** no controlador (redirecionamento). Isto respeita exatamente a restrição do enunciado (nada de named pipes entre controlador e veículo).

(C) Cliente ↔ Veículo

- O veículo cria um FIFO próprio "*./tmp/vec_<PID>_req*" e envia esse nome ao cliente pelo FIFO privado do cliente (que o controlador usa para comunicar com cada cliente privado).
- O cliente guarda esse nome e usa-o para "entrar <destino>" e depois "sair".

```
>>> agendar 95 vai 20
  [CONTROLADOR-SV] Servico (id: 2) agendado para a hora 95. Dist: 20km.

>>>    [CONTROLADOR-SV] Veiculo chegou ao local. Responda com 'entrar <desti
o pipe ./tmp/vec_30787_req
  [CLIENTE] PRONTO PARA EMBARQUE. Digite 'entrar <destino>'.

>>> entrar go

>>>    [CONTROLADOR-SV] Viagem COMEÇOU! Desfrute da paisagem.

>>>    [CONTROLADOR-SV] A sua viagem esta a decorrer: 10% concluido (2 km).
```

3. Estruturas internas e modelo de estado

O controlador mantém:

- lista de utilizadores ativos (username + PID);
- lista de serviços (id, hora_inicio, distância, estado, pid_cliente, pid_veiculo, distância_percorrida, etc.);
- frota ativa (entradas com pid_veiculo, FD do pipe de telemetria e thread associada);
- contador total de km (incrementado com base nas mensagens PROGRESSO).

Os serviços evoluem tipicamente pelos estados: AGENDADO → EM_VIAGEM → CONCLUÍDO/CANCELADO (e o admin pode cancelar inclusive em execução). Esta abordagem está alinhada com os comandos e regras do enunciado.

```
--- Serviços Agendados (2/100) ---
 ID 1: Cliente ronaldo | Hora: 500 | Distância: 20 | ESTADO: AGENDADO
 ID 2: Cliente pedro | Hora: 95 | Distância: 20 | ESTADO: EM VIAGEM
--------------------------------
```

4. Implementação

4.1. Organização do projeto

common.h: constantes de FIFOs, enums de estado, structs de mensagens, etc.

controlador.c: servidor + relógio + gestão de frota e telemetria.

cliente.c: interface e encaminhamento de comandos para controlador/veículo.

veiculo.c: simulação de viagem e telemetria.

makefile: compilação separada e target final. (clean remove binários e objetos.)

```
pedro@debianSO: ~/SO/PraticalWork/so_2526_tp_PedroSaraiva_2023146226_Rub...
pedro@debianSO:~/SO/PraticalWork/so_2526_tp_PedroSaraiva_2023146226_RubenCosta_2024132
692$ ls
cliente.c  common.h  controlador.c  makefile  tmp  veiculo.c
pedro@debianSO:~/SO/PraticalWork/so_2526_tp_PedroSaraiva_2023146226_RubenCosta_2024132
692$ make
gcc  -c controlador.c
gcc  controlador.o -o controlador -pthread
gcc  -c cliente.c
gcc  cliente.o -o cliente
gcc  veiculo.c -c
gcc  veiculo.o -o veiculo
echo "Aplicações compiladas com sucesso!"
Aplicações compiladas com sucesso!
# rm -f *.o # para não ter de estar sempre a apagar os objetos depois
pedro@debianSO:~/SO/PraticalWork/so_2526_tp_PedroSaraiva_2023146226_RubenCosta_2024132
692$ ls
cliente    cliente.o  controlador    controlador.o  tmp      veiculo.c
cliente.c  common.h   controlador.c  makefile       veiculo  veiculo.o
pedro@debianSO:~/SO/PraticalWork/so_2526_tp_PedroSaraiva_2023146226_RubenCosta_2024132
692$
```

4.2. Concorrência no controlador: threads reais

No controlador existem:

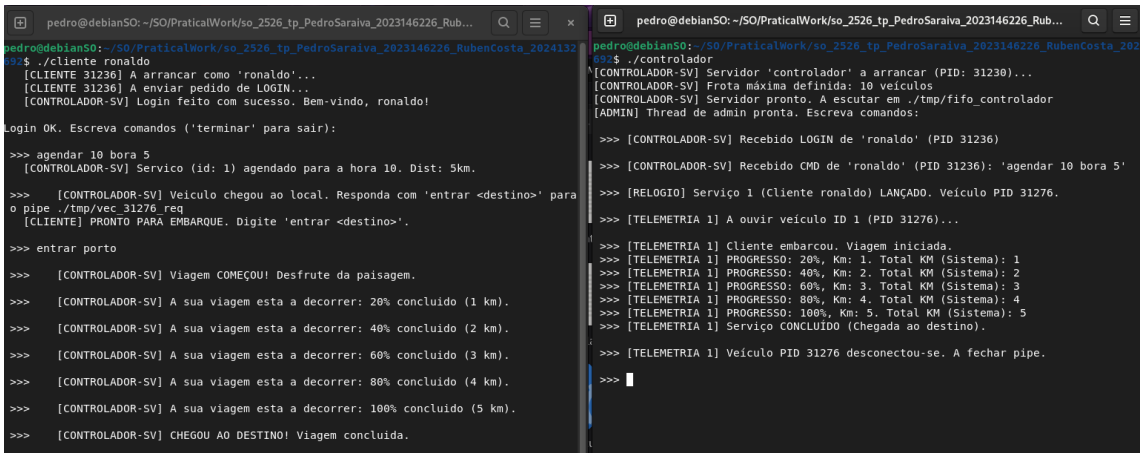
1. **Thread de clientes**: bloqueia em read() no FIFO público e processa pedidos de login e comandos.
2. **Thread de admin**: lê comandos do stdin (utiliz, listar, frota, cancelar, km, hora, terminar).
3. **Thread do relógio**: incrementa hora_simulada e lança veículos à hora marcada com fork()+exec().
4. **Threads de telemetria (veículos)**: uma por veículo ativo, lê do pipe anónimo (stdout do veículo) e atualiza serviço/contador km e notifica cliente.

Isto respeita a restrição "sem select no controlador", porque a simultaneidade é obtida com threads (stdin, FIFO público e telemetrias) em vez de multiplexagem com select().

4.3. Mutexes: onde protegem e o que foi evitado

Foram usados dois mutex principais:

- **mutex_utilizadores**: protege a lista de utilizadores ativos (login / listagem / remoção).
- **mutex_servicos**: protege a lista de serviços, frota ativa e variáveis associadas (hora_simulada, km totais, etc.).

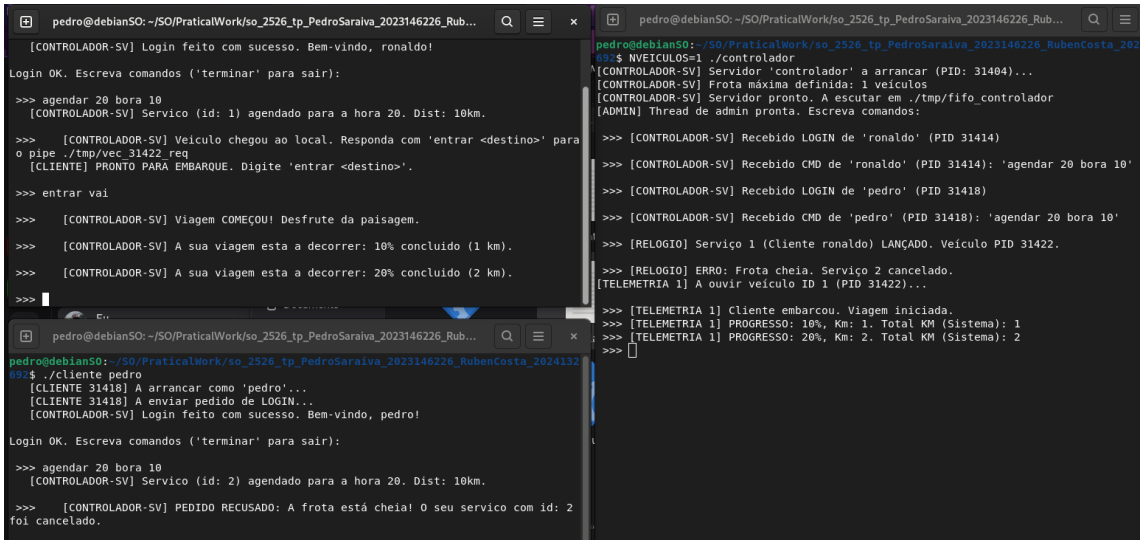


4.4. Variável de ambiente NVEICULOS

O enunciado define que a frota máxima vem da variável de ambiente NVEICULOS e que não pode haver mais do que N veículos em execução.

A estratégia implementada:

- ler getenv("NVEICULOS") no arranque do controlador;
- converter com atoi e validar limites;
- usar nveiculos_max para impedir lançar novos veículos quando a frota está cheia.



5. Tabela de funcionalidades requeridas

Funcionalidade do enunciado	Estado	Observações
Só 1 controlador em execução	Cumprido	Apenas permite ter um controlador em execução.
Comunicação cliente-controlador por named pipes	Cumprido	Modelo FIFO público + FIFO privado por cliente (padrão típico).
Comunicação controlador-veículo por args + stdout redirecionado	Cumprido	execlp com argumentos e stdout pelo pipe anónimo.
Sem select no controlador	Cumprido	Concorrência feita com threads.
NVEICULOS limita veículos em execução	Cumprido	Enunciado define NVEICULOS. A verificação "frota cheia" existe.
Tempo simulado como contador	Cumprido	hora_simulada incrementa de 1 em 1 segundo.
Cliente: agendar/cancelar/consultar	Cumprido	Comandos do cliente descritos no enunciado.
Cliente: entrar/sair para veículo	Cumprido	Handshake e envio pelo FIFO do veículo.
Veículo: progresso a cada 10%	Cumprido	Mensagens PROGRESSO + fflush.
Veículo: SIGUSR1 cancela e termina	Cumprido	Mensagem de cancelamento é tratada e mostrada ao cliente.
Controlador: km total do sistema	Cumprido	Atualiza com base em PROGRESSO e mostra via comando km.
Admin: utiliz/listar/frota/cancelar/km/hora/terminar	Cumprido	Conjunto de comandos exigido no enunciado.

6. Conclusões

O trabalho desenvolvido permitiu aplicar os principais conceitos da unidade curricular de Sistemas Operativos, incluindo processos, threads, comunicação entre processos e sincronização. A arquitetura baseada na separação entre cliente, controlador e veículo revelou-se adequada e coerente com o problema proposto.

A comunicação foi implementada através de named pipes, pipes anónimos, e outras formas de comunicação, respeitando as restrições do enunciado. O controlador recorre a múltiplas threads para tratar pedidos de clientes, comandos do administrador, simulação do tempo e leitura de telemetria, evitando soluções baseadas no uso de select().

O sistema suporta o agendamento, consulta e cancelamento de serviços, bem como o acompanhamento e interrupção das viagens. No conjunto, a solução apresentada cumpre os objetivos definidos no enunciado e demonstra uma aplicação correta dos mecanismos fundamentais de sistemas operativos.